

Bit Flip Learning: Direct Updates and Boolean Greedy Search

Facundo Gomez Prates

October 3, 2026 — theory and experiments

Abstract

This manuscript distinguishes three implemented forms of Bit Flip Learning. A direct-state MUX graph applies $S' = S \oplus (C \wedge U)$ to active selectors after each output comparison. A layered MUX model uses single-bit greedy search; joint path masks and weighted scores are reported separately as exploratory variants of that model. The routed binary model’s principal greedy rule gates candidate masks of at most three bits by active features and selects a strict lexicographic improvement in route error, forgetting, and change cost. The direct graph’s partial-prefix route increased route diversity without improving held-out accuracy in its lookup experiment. In a separate 50-seed binary-threshold comparison, the routed model averaged 90.31% test accuracy versus 99.48% for a 16-unit neural network, while storing 256 versus 1544 bytes of parameter state. The routed model uses fixed features tailored to that task. These results characterize the specified implementations and do not imply a generalization guarantee.

Keywords: Bit Flip Learning, multiplexers, Boolean greedy search, binary state, active paths.

1 Motivation and scope

A multiplexer selects one of two signals using a binary selector. The library contains three related but distinct models. The direct-state graph treats changes to selector state as its local learning operation. Its law, written for a selector S , is

$$S' = S \oplus (C \wedge U), \quad S, C, U \in \{0, 1\}. \quad (1)$$

In a supervised output comparison, $C = \text{OUT} \oplus \text{TARGET}$; U records whether a selector participated in computing that output. The state toggles exactly when $C = U = 1$; if either bit is zero, it stays the same. In this direct-state formulation, S is the selector state itself. The layered greedy model has a separate inversion parameter m and an accept-or-revert test. The routed Boolean model gates candidate changes by active features and also tests them before acceptance. Equation (1) describes the direct graph’s update and the routed model’s *effective candidate mask*; it does not by itself specify either greedy acceptance rule.

We first formalize the direct-state supervised per-output core: a forward pass supplies OUT, a target supplies TARGET, and a backward trace supplies USED. To define a runnable model beyond one comparison, we state a particular schedule for several examples and outputs, a causal count route, and a partial-prefix extension. We then describe the layered MUX model, identify its later joint-mask experiments, and define the routed Boolean greedy rule with its matched held-out comparison. The weighted J scores in the MUX experiments are not the routed model’s acceptance rule. Routing choices, feature maps, and candidate searches are additional design decisions; they do not follow from the local Boolean equation alone.

2 A graph model with direct selectors

Let $G = (V, E)$ be an acyclic graph of input nodes, optional constant nodes, and multiplexer nodes. Each multiplexer v has data predecessors $a(v)$ and $b(v)$. Its output for input $x \in \{0, 1\}^p$

is

$$z_v(x; S) = \text{MUX}(z_{a(v)}(x; S), z_{b(v)}(x; S), \sigma_v(x; S)), \quad \text{MUX}(A, B, \sigma) = (\neg\sigma \wedge A) \vee (\sigma \wedge B). \quad (2)$$

The symbol σ_v denotes the value supplied to the selector input of node v . The basic graph permits two explicitly distinguished kinds of selector:

$$\sigma_v(x; S) = S_v \quad \text{for an adaptive node } v, \quad (3)$$

$$\sigma_v(x; S) = z_{d(v)}(x; S) \quad \text{for a signal-controlled node } v. \quad (4)$$

An adaptive node reads its own mutable selector state $S_v \in \{0, 1\}$ *directly*. A signal-controlled node reads an input or earlier graph signal through $d(v)$; it needs no separate inversion parameter. The mutable vector S consists of the adaptive selector states. An input-dependent selector can therefore be present without representing its learning as an XOR mask on top of it. This minimal distinction leaves open richer ways to make a selector context dependent.

The graph may share predecessors, making it a directed acyclic graph rather than necessarily a tree. Composing MUXes in multiple layers gives a larger network; a tree of seven 2:1 MUXes also implements an 8:1 choice. A set of q designated outputs defines $f_S : \{0, 1\}^p \rightarrow \{0, 1\}^q$. For a supervised input x_i and output bit j , let $C_{i,j}$ be its CHANGE bit and $U_{i,j,v}$ the USED bit of an adaptive node v , defined below. Its per-output update is

$$S_v^{(t+1)} = S_v^{(t)} \oplus (C_{i,j}^{(t)} \wedge U_{i,j,v}^{(t)}). \quad (5)$$

The indices identify the example, output bit, and direct selector state; they do not introduce a flip-mask variable. For this output comparison a change occurs only when $C_{i,j} = U_{i,j,v} = 1$. Once the state changes, the branch selected by that adaptive node may differ on other inputs as well. If the intended model instead uses context-specific states, their indexing and persistence require an additional definition.

2.1 Library realization

The library's `bfl::Graph` class uses this DAG model directly. `add_adaptive_mux` creates a MUX with a direct binary selector state, while `add_signal_mux` creates one whose selector is an earlier signal. Constants are dedicated nodes. This class trains adaptive states by Eq. (5); its routing convention and prefix parameter are defined below. The later sections define the separate `bfl::GreedyModel` and `bfl::RoutedBinaryModel` implementations.

3 Active paths and USED

For input x_i and output bit j , define the active dependency subgraph $D_j(x_i; S)$ recursively from output j . For each reached MUX, include the dependency of its selector, if it has one, and include only the data predecessor chosen by σ_v . A signal-controlled selector in Eq. (4) contributes its predecessor $d(v)$; an adaptive selector in Eq. (3) reads S_v directly. A shared node is counted once. Define

$$U_{i,j,v}(S) = \mathbf{1}\{v \in D_j(x_i; S)\}. \quad (6)$$

The USED bits can be computed explicitly from a forward pass followed by reverse traversal. For a fixed (i, j) , initialize $U_{i,j,v} = 0$ everywhere and set the designated output node's bit to one. Process marked MUXes in reverse topological order. If v is marked, send its mark to the selector predecessor when one exists, send it to $a(v)$ only when $\sigma_v = 0$, and send it to $b(v)$ only when $\sigma_v = 1$. Merge marks arriving along shared paths with OR. Algebraically, each marked MUX contributes

$$U_{i,j,a(v)} \leftarrow U_{i,j,a(v)} \vee (U_{i,j,v} \wedge \neg\sigma_v), \quad (7)$$

$$U_{i,j,b(v)} \leftarrow U_{i,j,b(v)} \vee (U_{i,j,v} \wedge \sigma_v), \quad (8)$$

$$U_{i,j,d(v)} \leftarrow U_{i,j,d(v)} \vee U_{i,j,v} \quad \text{if } v \text{ has a selector predecessor.} \quad (9)$$

On an 8:1 tree whose address bits come directly from inputs, the data trace visits one MUX at each of its three levels; a selector computed by another MUX also brings that selector’s dependencies into USED. This is a causal-path mark, independent of whether an implementation eagerly evaluates inactive branches. It is the U gate in Eq. (1), but it does not by itself say that changing S_v improves the output.

Proposition 1 (Inactive adaptive state). *For an adaptive node v , if $U_{i,j,v}(S) = 0$, changing only S_v leaves $f_S(x_i)_j$ unchanged.*

Proof. The value of output j depends only on its active selector and chosen data dependencies. Since v is absent from that dependency graph, changing its state cannot change a dependency already used to compute the output. Induction along the acyclic order leaves each relevant value, and hence the output, unchanged. $\square$

The converse need not hold: a used node may choose branches with equal values, or another part of the graph may cancel its effect. USED is therefore a way to restrict possible changes, not a complete credit-assignment mechanism.

4 Computing CHANGE and updating selectors

For supervised examples $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, with $y_i \in \{0, 1\}^q$, compute the output $o_i = f_S(x_i)$. The CHANGE bit for output j is its binary mismatch:

$$C_{i,j} = o_{i,j} \oplus y_{i,j} = \text{OUT}_{i,j} \oplus \text{TARGET}_{i,j}. \quad (10)$$

Thus $C_{i,j} = 0$ for a correct bit and $C_{i,j} = 1$ for a wrong bit. For one (i, j) , compute all $U_{i,j,v}$ from the same pre-update forward pass, then update every adaptive selector on that output’s traced graph from the same state snapshot using Eq. (5). Equivalently, $\Delta S_v = C_{i,j} \wedge U_{i,j,v}$ and $S'_v = S_v \oplus \Delta S_v$ for every adaptive v . A correct output changes none of these selectors. An incorrect output changes only adaptive selectors on its USED path. No extra inversion mask F is part of this rule.

This describes a direct per-output update: after computing C and U , the corresponding selector states change without first computing a scalar loss or testing alternative parameter settings. Several errors still require a scheduling convention. The following choice makes repeated presentations unambiguous while retaining the original one-comparison law.

4.1 An explicit schedule for several examples and outputs

Fix an order of the examples $i = 1, \dots, n$ and output bits $j = 1, \dots, q$. Number their lexicographic comparisons by $k = (i - 1)q + j$, and let $S^{[k]}$ be the whole selector vector before comparison k . Compute a *fresh* forward output and backward trace at $S^{[k]}$, then simultaneously update all adaptive nodes for that comparison:

$$\begin{aligned} C^{[k]} &= f_{S^{[k]}}(x_i)_j \oplus y_{i,j}, \\ U_v^{[k]} &= U_{i,j,v}(S^{[k]}), \\ S_v^{[k+1]} &= S_v^{[k]} \oplus (C^{[k]} \wedge U_v^{[k]}) \quad \text{for every adaptive } v. \end{aligned} \quad (11)$$

One pass over all nq comparisons is an epoch; another epoch starts from its final state. No cross-example error is summed, voted on, or converted to a scalar objective. In particular, all traces for a single comparison come from its pre-update snapshot, whereas the next comparison sees the updated graph. The schedule is a *proposed completion*: the core equation alone neither selects an example order nor equates this online process with a batch update. Permuting examples or output bits can change the state trajectory.

Even perfect single-example correction does not imply convergence across examples. Take the one-node graph $f_S(x) = \text{MUX}(0, 1, S) = S$ and two presentations whose targets are respectively zero and one. Starting at $S = 0$, the first comparison leaves $S = 0$ and the second changes it to $S = 1$. On the next epoch the first changes it back to zero and the second changes it to one. Thus the epoch-end state remains one while the first target is repeatedly wrong. No selector state can satisfy both targets in this graph. This is an explicit limit of the online convention, not a failure of MUX representational completeness with a larger graph.

5 Examples and expressivity

The simplest direct-state example is $z_v = \text{MUX}(0, 1, S_v) = S_v$. For a single target $y = 1$ with $S_v = 0$, the forward output is zero, so $C = 0 \oplus 1 = 1$. The output node is USED, so $U = 1$ and Eq. (5) sets $S'_v = 0 \oplus (1 \wedge 1) = 1$. If that node were not on the active route to the chosen output, $U = 0$ would prevent its update. There is no dynamic selector signal and no additional inversion bit.

Signal-controlled MUXes can construct input-dependent functions. For example, if $\neg x_2$ is available from a Boolean subgraph, then

$$\text{MUX}(x_2, \neg x_2, x_1) = x_1 \oplus x_2. \quad (12)$$

This example explains the expressive role of an input-driven selector. Equation (12) contains no adaptive state, so it is not by itself an example of learning XOR with CHANGE. A trainable state could select between two already constructed functions, but learning the required subgraphs or topology is a separate problem.

Proposition 2 (Representational completeness of the MUX basis). *With input-controlled selectors, constants, and a graph whose size may grow with the target, any finite Boolean function $g : \{0, 1\}^p \rightarrow \{0, 1\}$ has a multiplexer representation.*

Proof. Shannon decomposition gives $g = \text{MUX}(g|_{x_k=0}, g|_{x_k=1}, x_k)$. Recursively decompose each cofactor until the leaves are constants. The resulting finite graph consists of signal-controlled MUXes and may share identical subgraphs. $\square$

This is a property of the computational basis, independent of the CHANGE law. It does not establish that a fixed graph can express every target or that direct-selector updates discover a useful representation. For a fixed graph with N adaptive state bits, at most 2^N distinct parameter settings exist, possibly representing fewer than 2^N distinct functions.

Indeed, representability does not even guarantee success on one repeated target for the specified update. Consider two adaptive states S_1, S_2 and the graph

$$z_2 = \text{MUX}(0, 1, S_2), \quad w = \text{MUX}(1, 0, z_2), \quad o = \text{MUX}(z_2, w, S_1) = S_1 \oplus S_2. \quad (13)$$

The first two MUXes supply S_2 and its complement; w has a signal-controlled selector. Both adaptive nodes are USED regardless of the current states: the root reads S_1 , while its chosen branch depends on z_2 . For target $o = 1$, initialization $(S_1, S_2) = (0, 0)$ gives $C = 1$ and toggles both states to $(1, 1)$, leaving $o = 0$. Repeating the presentation toggles both back. The graph represents the target at $(0, 1)$ or $(1, 0)$, yet this trajectory never reaches it. This example proves only a failure of this update from this initialization; it does not exclude other graphs or update schedules.

6 Count routing from effective prior changes

Definition. Order graph nodes topologically, and let $v_0, \dots, v_{m-1}$ be the MUXes in that order. For a supervised comparison k , define the event at MUX v_i by

$$A_i^{[k]} = C_i^{[k]} \wedge U_i^{[k]}, \quad C_i^{[k]} = \begin{cases} C_i^{[k]}, & v_i \text{ adaptive,} \\ 0, & v_i \text{ signal-controlled.} \end{cases} \quad (14)$$

A signal-controlled MUX has no mutable selector state, so it cannot produce a state modification. For adaptive MUXes, $A_i = 1$ precisely when the output is wrong and that MUX is USED. The route for MUX v_j is

$$N_j^{[k]} = \sum_{i < j} A_i^{[k]}, \quad R_j^{[k]} = \text{bin}(N_j^{[k]}). \quad (15)$$

Here N_j is a nonnegative integer and bin is its ordinary base-two representation $(0, 1, 10, 11, \dots)$. It is not a bitmask of earlier MUX positions. The current MUX’s own event does not enter its route. Equivalently, traverse MUXes in topological order, assign $R_j \leftarrow \text{bin}(N)$, then set $N \leftarrow N + A_j$, starting from $N = 0$. The library stores the integer N_j as the route key; `binary_route` exposes its binary digits. For example, preceding event patterns 1010 and 0101 both give $N_j = 2$ and $R_j = 10_2$, whereas 1101 gives $N_j = 3$ and $R_j = 11_2$. At the same MUX, equal counts select the same state bank. This is structural reuse; equality of routes does not assert equality of external inputs X , targets Y , or active graph paths. A route describes prior internal processing.

Causal timing and update. `CHANGE` depends on a target and `USED` depends on the forward-selected paths, so the event vector for a comparison is only known after its forward and backward passes. Let $A^{[k-1]}$ be the events recorded from the previous comparison; initialize it to all zero. Comparison k evaluates each MUX with $N_j^{[k-1]}$ from Eq. (15). An adaptive MUX reads the state $S_{v_j}^{(N_j^{[k-1]})}$ for that route. After computing $C^{[k]}$ and all $U^{[k]}$ from this same snapshot, it updates each addressed state exactly by

$$S_{v_j}^{(N_j^{[k-1]})'} = S_{v_j}^{(N_j^{[k-1]})} \oplus (C^{[k]} \wedge U_{v_j}^{[k]}), \quad (16)$$

and records $A^{[k]}$ for the next comparison. Routing chooses which direct state is read; it never substitutes for Eq. (1). New route banks start at the MUX’s initial state and are materialized only when changed. The graph’s route is computed in one topological pass by adding each preceding event after assigning the current MUX’s route. No overflow occurs while the count is bounded by the number of graph MUXes.

Training traverses examples and outputs in a fixed order for each epoch, with a fresh forward pass for each comparison. Inference has no target from which to derive a new `CHANGE` bit. It therefore reads the last recorded event vector without modifying it; callers may explicitly restore a valid event vector with `set_routing_events`. This is a model-context convention, not an input-conditioned router. A model requires its graph, route-specific states, and final event vector to reproduce predictions. The library currently has no persistence format; copying a `Graph` preserves these values in memory.

Hypothesis. Sharing a state when counts coincide may exert compression or regularization pressure, because several event patterns update the same route bank. This is a hypothesis about learning behavior, not a consequence of the route equation. Distinct patterns can also interfere at a shared state, and the online rule can oscillate. Held-out measurements are required before any generalization claim.

7 Partial-prefix route experiment

The count route deliberately merges different effective-event positions. To retain a bounded amount of positional information, let $k \in \{0, \dots, 64\}$ and define, for MUX v_j ,

$$P_{j,k} = \sum_{i=0}^{\min(j,k)-1} A_i 2^{\min(j,k)-1-i}, \quad R_{j,k} = (N_j, P_{j,k}). \quad (17)$$

The bits of $P_{j,k}$ are the first k MUX event bits that precede v_j , in topological graph order; signal-controlled MUXes contribute zero bits. At a fixed node their prefix length is fixed. With $k = 0$, $P_{j,0} = 0$ and the route is exactly the count route. For $k = 2$, preceding patterns 1010 and 0101 have the same count two but prefixes 10_2 and 01_2 , so they address different states. Patterns 1010 and 1001 still share a route. The implementation stores the pair as a `RouteKey`; `Graph(k)` selects k , and the default constructor uses $k = 0$. The causal timing, USED trace, and update law remain those of Eq. (16), with $R_{j,k}$ replacing the count address. This is a deliberately partial bitmask: events beyond the first k positions contribute to the count but not to the prefix.

8 Measured benchmark result

Experimental result. The reproducible C++ benchmark enumerates the binary domains of width 4, 6, 8, shuffles them with seeds 1, 2, 3, uses the first half for online training and the next quarter as disjoint held-out inputs, and measures parity and majority output bits. BFL uses two full lookup trees: adaptive leaves select between constants, and signal-controlled internal MUXes select by input bits. We compare $k = 0, 4, 8, 16$ with identical splits, initialization, and update order, for one and three epochs. The majority baseline stores the most frequent training bit per output. Table 1 gives one-epoch arithmetic means from `results/run_2026-09-30-prefix/benchmark.csv`; the CSV also includes per-seed timings, sizes, and other metrics. Distinct routes

bits	k	train acc.	test acc.	routes	USED addresses
4	0	.875	.375	2.00	16
4	4	.875	.375	4.67	16
4	8	.875	.375	8.67	16
4	16	.854	.375	16.67	16
6	0	.818	.510	2.00	64
6	4	.818	.510	3.67	64
6	8	.818	.510	7.67	64
6	16	.818	.510	15.67	64
8	0	.861	.448	2.00	256
8	4	.861	.448	3.67	256
8	8	.861	.448	6.33	256
8	16	.861	.448	14.33	256

Table 1: Mean bit accuracies, distinct route keys, and distinct adaptive selector addresses reached by USED traces across three seeds after one epoch.

rise with k , confirming that the prefix separates event patterns that count routing merges. Yet the number of adaptive $(v, R_{j,k})$ addresses actually reached by USED is unchanged at every width. The lookup tree assigns each input its own adaptive leaf, so this architecture gives different training inputs little opportunity to compete for the same addressed state. After three epochs every BFL variant reached training bit accuracy 1.000 at all three widths; held-out accuracy remained .375, .510, and .448, respectively, for every k . Inference keeps the final event vector as a common context for all held-out inputs. These measurements show route diversity without a held-out gain and do not validate a generalization hypothesis for partial-prefix routing.

9 Layered greedy MUX model

The library also provides `bfl::GreedyModel`, a C++ implementation of a layered random MUX network. This model has a different selector parameterization and training rule from `bfl::Graph`. For node v , let $a(v)$ and $b(v)$ be data predecessors, $d(v)$ a selector predecessor, and $m_v \in \{0, 1\}$ a learned inversion bit. Each predecessor is an input column or a node in the immediately preceding layer. The node output is

$$z_v(x; m) = \text{MUX}(z_{a(v)}(x; m), z_{b(v)}(x; m), z_{d(v)}(x; m) \oplus m_v). \quad (18)$$

The topology stays fixed during one training run; training changes only m . A constant input column, when supplied, gives the network access to a fixed one. Each node's three source indices and state bit are stored in one 64-bit word. Forward evaluation packs up to 64 examples into each machine word; this changes the execution layout, not Eq. (18).

For a training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ with q output bits, define the global Hamming objective

$$L_{\mathcal{D}}(m) = \sum_{i=1}^n \sum_{j=1}^q (f_m(x_i)_j \oplus y_{i,j}). \quad (19)$$

For each currently wrong output bit, a USED trace marks its output MUX, the selector dependency, and the selected data dependency, recursively. The union of marked MUXes over the training set supplies candidate nodes. At the start of each epoch the candidates may be shuffled and capped. For each candidate v in sequence, the implementation proposes $m' = m \oplus e_v$, recomputes $L_{\mathcal{D}}(m')$ on the entire training set, and keeps the proposal only if

$$L_{\mathcal{D}}(m') < L_{\mathcal{D}}(m). \quad (20)$$

Otherwise it restores the old bit. A later candidate is scored against the state left by earlier accepted flips. Candidate traces are refreshed at the next epoch. This is a global, strictly improving, single-bit search; Eq. (1) is not its update law.

Proposition 3 (Monotone training error for the greedy model). *Within one training run, the global Hamming error cannot increase after a candidate test. The number of accepted single-bit flips is at most the initial Hamming error.*

Proof. An accepted flip decreases the nonnegative integer $L_{\mathcal{D}}$ by at least one; a rejected flip restores the preceding state. Thus every accepted flip consumes at least one unit of the initial error, and all other tests leave it unchanged. $\square$

This proposition concerns the original single-bit rule and the training set only. Candidate limits, topology, and local minima can prevent zero training error; monotonic training error does not imply held-out improvement.

9.1 Exploratory joint MUX proposals and weighted scores

The following variants are later experiments on `bfl::GreedyModel`; they do not define the principal greedy rule of `bfl::RoutedBinaryModel`. At the current MUX state m , an incorrect output (i, j) has $C_{i,j} = f_m(x_i)_j \oplus y_{i,j} = 1$. Trace the output MUX, each reached selector predecessor, and each selected data predecessor. Let $U_{i,j,v}(m)$ mark the MUX state bits reached by that trace. The candidate mask and trial state are

$$F_{i,j,v}(m) = C_{i,j}(m) \wedge U_{i,j,v}(m), \quad m_v^{\text{trial}} = m_v \oplus F_{i,j,v}(m). \quad (21)$$

This is a projection of one wrong *output* bit onto a set of MUX *state* bits; it does not identify their indices or claim that flipping every reached bit corrects the output. Duplicate masks from

different examples are tested only once per candidate collection. After an accepted mask, traces and candidate masks are regenerated at the new state.

For each example, define $U_{i,v}^\cup(m) = \bigvee_{j=1}^q U_{i,j,v}(m)$, tracing all outputs whether correct or incorrect. Let N be the number of mutable MUX bits. The measured training error, usage, and unused counts are

$$E_{\mathcal{D}}(m) = \sum_{i,j} C_{i,j}(m), \quad T_{\mathcal{D}}(m) = \sum_{i,v} U_{i,v}^\cup(m), \quad Q_{\mathcal{D}}(m) = nN - T_{\mathcal{D}}(m). \quad (22)$$

The implementation evaluates each trial on the *whole* training set and accepts it only if a chosen integer objective strictly decreases:

$$\begin{aligned} J_{a,b}^{\text{used}}(m) &= aE_{\mathcal{D}}(m) + bT_{\mathcal{D}}(m), \\ J_{a,b}^{\text{unused}}(m) &= aE_{\mathcal{D}}(m) + bQ_{\mathcal{D}}(m), \\ m' &= \begin{cases} m^{\text{trial}}, & J(m^{\text{trial}}) < J(m), \\ m, & \text{otherwise.} \end{cases} \end{aligned} \quad (23)$$

The tested USED variant sets $(a, b) = (10, 1)$. A validation search over NOT(USED) variants selected (17, 1) within the stated search protocol; the software permits other integer weights. That score minimizes $E + \lambda Q$ with $\lambda = b/a = 1/17$. Since nN is constant, $J_{1,1}^{\text{unused}}$ orders states like $E - T$: a gain in active MUXes can offset an extra error. These J scores apply only to the exploratory layered-MUX proposals in this subsection. They are not substitutions for the routed lexicographic rule below.

Proposition 4 (Monotone masked training score). *With nonnegative integer weights, each accepted joint proposal strictly lowers the chosen J on the fixed training set. This does not imply that $E_{\mathcal{D}}$ decreases when $b > 0$.*

Proof. The rule in Eq. (23) retains a trial only under strict inequality and otherwise restores m . The usage term can change in the opposite direction from error, so a decrease in their weighted sum alone gives no ordering of $E_{\mathcal{D}}$. $\square$

Candidate and epoch limits may stop before a local minimum, and a local minimum for one topology does not imply a globally optimal Boolean function.

9.2 Matched generalization comparison

We compare this greedy model, a direct-state lookup **Graph**, and a one-hidden-layer neural network (MLP) on the same splits. Each example has eight varying input bits and a constant-one column. The targets are OR or XOR of the first two bits, or majority of the first three; the remaining varying bits are irrelevant to the label. For each of 20 seeds, the 256 distinct eight-bit inputs are shuffled into 128 training, 64 validation, and 64 held-out test examples. The test split is used only after training and topology selection.

The greedy model has three hidden layers of 64 MUXes and one output MUX. It runs at most 100 epochs, tests at most 128 candidate nodes per epoch, and stops after 15 epochs without an accepted flip. We report the first random topology and the best of 16 independently trained topologies, choosing the latter by lowest validation error and then lowest training error. The MLP has 16 tanh hidden units, one sigmoid-equivalent binary output, and 1500 full-batch Adam epochs. The direct-state control has one adaptive lookup leaf per eight-bit input and runs three online epochs. Every model receives the same input columns and split. Table 2 gives arithmetic means over the 20 seeds from `results/run_2026-09-30-generalization/benchmark.csv`; `scripts/reproduce_generalization.sh` reproduces the comparison.

On OR2, the selected greedy model reaches .762 held-out accuracy versus 1.000 for the MLP under this protocol. This new result is not a reproduction of the previously reported approximate

task	model	train acc.	validation acc.	test acc.
OR2	greedy, first topology	.797	.685	.668
OR2	greedy, selected of 16	.880	.825	.762
OR2	direct-state lookup	1.000	.245	.245
OR2	MLP	1.000	1.000	1.000
Majority3	greedy, first topology	.773	.670	.637
Majority3	greedy, selected of 16	.834	.793	.733
Majority3	direct-state lookup	1.000	.477	.503
Majority3	MLP	1.000	1.000	1.000
XOR2	greedy, first topology	.679	.557	.546
XOR2	greedy, selected of 16	.746	.697	.608
XOR2	direct-state lookup	1.000	.498	.488
XOR2	MLP	1.000	1.000	1.000

Table 2: Mean bit accuracies on matched training, validation, and held-out test splits. Topology selection uses validation data only.

80% versus 100% comparison: its original dataset and split are unavailable. Selection improves the greedy model’s mean test score over its first topology on all three tasks, while its absolute accuracy depends on the task. The direct-state control memorizes observed input addresses, so its test score does not isolate the effect of Eq. (1). The models also differ in topology, training rule, and search budget; matched data splits alone do not establish an algorithm-wide ranking.

9.3 Exploratory masked-rule comparison and weight selection

The joint-mask runs use the same 20 matched train/validation/test splits and 16-topology validation selection. A validation-only sweep evaluates $a = 1, \dots, 128, 256, 512, 1024, 25000$ with $b = 1$ in $J_{a,b}^{\text{unused}}$. Over the 60 task/seed validation results, $a = 17$ gives 977 errors among 3840 validation predictions; $a = 8$ and $a = 9$ each give 978. Test targets are withheld until after choosing 17. This one-error validation lead does not establish a stable universal weight. Table 3 reports held-out means from `results/run_2026-10-02-weight-search/benchmark.csv`. Pooled over

task	single-bit greedy	joint mask, $17E + Q$	MLP (16)
OR2	.762	.752	1.000
XOR2	.608	.613	1.000
Majority3	.733	.702	1.000

Table 3: Mean held-out accuracy over 20 matched splits per task. Greedy rows select among 16 trained topologies using validation data.

the 60 task/seed results, the joint rule at $a = 17$ reaches 68.88% test accuracy versus 70.10% for the original single-bit greedy; both MLP variants in the extended experiment reach 100%. The joint rule changes proposal size and acceptance score together, so these rows do not isolate a usage penalty. The search examined this finite weight grid and these tasks; its selected value is not a theoretical optimum.

10 Routed Boolean greedy model and its principal rule

The third implementation, `bfl::RoutedBinaryModel`, is a signed binary-feature classifier rather than a layered MUX graph. Its official greedy choice is the lexicographic rule defined in this section; it does not use the weighted J scores of the preceding MUX experiments. Let $x \in \{0, 1\}^{10}$ and define sixteen fixed features

$$\phi_0(x) = 1, \quad \phi_{i+1}(x) = x_i \ (0 \leq i < 10), \quad \phi_{11+i}(x) = x_{2i} \wedge x_{2i+1} \ (0 \leq i < 5). \quad (24)$$

The first five raw input bits address one of 32 routes, $r(x) = \sum_{i=0}^4 2^i x_i$. Each route r has a mutable 16-bit state s_r . Feature h contributes +1 when $s_{r,h} = 1$ and -1 when $s_{r,h} = 0$, but only if $\phi_h(x) = 1$. The implemented prediction is

$$f_s(x) = \mathbf{1} \left\{ \sum_{h=0}^{15} (2s_{r(x),h} - 1) \phi_h(x) \geq 0 \right\}. \quad (25)$$

A zero score predicts one. The route function and feature map remain fixed during training. In the C++ representation, each route state occupies one 64-bit word although only its low sixteen bits participate in prediction.

10.1 Candidate update and lexicographic selection

For a presented example (x, y) , USED is the active feature mask $u_h(x) = \phi_h(x)$. Let $\Delta \in \{0, 1\}^{16}$ be a proposed CHANGE mask with Hamming weight at most three; the all-zero mask is included. The official effective mask and trial state for its route are

$$d_h = \Delta_h \wedge u_h(x), \quad s_{r(x),h}^{\text{trial}} = s_{r(x),h} \oplus d_h, \quad (26)$$

with other route states untouched. Here CHANGE means a *candidate feature-state mask*, whereas C in Eq. (10) means an *output mismatch*. The two bits have different index sets and roles. The unmasked $d = \Delta$ variant is an experimental control; it can change state bits inactive for the presented x that matter on other examples.

Let $I_r = \{i : r(x_i) = r\}$ index the training memory of route r , and write $e_i(s) = f_s(x_i) \oplus y_i$. At each presented example the learner evaluates a trial against *all* examples in its route. Its three Boolean counts are

$$\begin{aligned} E_r(s') &= \sum_{i \in I_r} e_i(s'), \\ F_r(s', s) &= \sum_{i \in I_r} ((\neg e_i(s)) \wedge e_i(s')), \\ K(d) &= \sum_{h=0}^{15} d_h. \end{aligned} \quad (27)$$

E_r counts *forgetting*: examples that were correct before this candidate and wrong after it. For this presentation, write $S = s_{r(x)}$ for the addressed route state, $U = u(x)$ for its active-feature mask, and $S_\Delta = S \oplus (\Delta \wedge U)$. All other route states remain fixed when evaluating $E_r(S_\Delta)$ and $F_r(S_\Delta, S)$. The finite candidate set includes the no-op:

$$\mathcal{C} = \{\Delta \in \{0, 1\}^{16} : \text{popcount}(\Delta) \leq 3\}. \quad (28)$$

The principal greedy rule is the following lexicographic minimum, with the no-op preferred when its score ties a nonzero proposal:

$$\Delta^* \in \arg \min_{\Delta \in \mathcal{C}}^{\text{lex}} (E_r(S_\Delta), F_r(S_\Delta, S), \text{popcount}(\Delta \wedge U)).$$

(29)

The unchanged state scores $(E_r(S), 0, 0)$. The learner commits S_{Δ^*} only when its tuple is strictly smaller; otherwise it keeps S . This comparison supplies the KEEP/REVERT decision. It is a discrete greedy selection over a finite neighborhood, not gradient descent, and no weighted scalar J is evaluated for this routed model. Training shuffles the 600 presented examples each epoch and repeats for 20 epochs in the measured protocol. Candidate masks are tested anew at the current route state on every presentation.

Proposition 5 (Training-error monotonicity of the routed rule). *For a fixed training set and fixed route function, every accepted routed update strictly reduces total training error $E_{\mathcal{D}}(s) = \sum_r E_r(s)$. The number of accepted updates is at most its initial error count.*

Proof. The no-op score is $(E_r(s), 0, 0)$, while $F_r \geq 0$ and $K \geq 0$ for every trial. A trial can be strictly better than the no-op only if its first component E_r is smaller. An update changes exactly one route, so errors on all other routes stay fixed. Every accepted update therefore removes at least one error from the nonnegative integer $E_{\mathcal{D}}$. $\square$

The second and third score components choose between candidates that already improve route error; neither permits a same-error update against the no-op. Monotone training error gives no bound on unseen inputs, and the finite candidate neighborhood can end at a state with remaining training errors.

10.2 Matched binary-threshold experiment

The benchmark uses all $2^{10} = 1024$ distinct binary inputs, shuffled independently for each of 50 seeds. The first 600 form training and the remaining 424 form a disjoint test set. The target is a fixed threshold of terms included in Eq. (24):

$$\begin{aligned} g(x) = & -1 + x_0 + x_1 + (x_0 \wedge x_1) + (x_2 \wedge x_3) \\ & + (x_4 \wedge x_5) - x_6 - x_7 - (x_8 \wedge x_9), \\ y(x) = & \mathbf{1}\{g(x) \geq 1\}. \end{aligned} \tag{30}$$

All models receive the same raw inputs, labels, and split. The routed BFL model uses the fixed engineered features above. The older MUX control uses two hidden layers of width 16, at most 30 epochs, and at most 64 single-bit candidates per epoch. The neural control uses the ten raw bits, 16 tanh hidden units, and 500 full-batch Adam epochs with sigmoid/binary-cross-entropy training. No topology selection on a validation set is used in this experiment.

model	train acc.	test acc.	model bytes	train ms	infer ns
older MUX greedy	.6238	.5992	324	.124	90.55
routed Boolean greedy	.9601	.9031	256	194.42	9.70
MLP (16 units)	1.0000	.9948	1544	142.46	385.66

Table 4: Means over 50 matched seeds from `results/bfl_vs_ann.csv`. Stored bytes use each implementation’s stated representation; the older MUX count includes a 60-byte serialized header. Timings are measurements on the recorded machine.

The routed model’s mean test accuracy is 9.17 percentage points below the MLP’s; its 256 stored state bytes are 83.42% fewer than the MLP’s 1544 parameter bytes, and its mean measured inference time is 39.75 times lower. Its training is slower (194.42 versus 142.46 ms) and uses more estimated temporary memory (27,649 versus 4,760 bytes). The state-byte comparison excludes fixed feature/routing code and object overhead; inference timing excludes model construction and data preparation. The routed model’s mean test score ranges from .8561 to .9434 across seeds. These measurements do not isolate the Boolean update: routing, engineered features, topology, and search budgets differ. The target was built from the routed model’s own features, which favors that architecture. The MLP still has higher held-out accuracy in this experiment.

11 Relation to prior work and open questions

Boolean decision diagrams use shared graph representations of Boolean functions [1]. Cartesian genetic programming also studies computations represented as acyclic graphs [2]. Binary neural networks provide an empirical comparison point, although their state and training procedures

differ [3]. These antecedents do not resolve whether BFL’s direct CHANGE law yields an effective learning rule.

The online schedule and count route remove operational ambiguities for the direct-state model. They do not determine a best example order, resolve conflicting updates, or select a useful graph topology. The original layered MUX rule tests single-bit proposals against global training error; its later joint-mask experiments test separate weighted objectives. The routed model instead uses the lexicographic tuple in Eq. (29). Its training-error monotonicity does not imply held-out accuracy. The measurements above describe these implementations and tasks; broader claims require other datasets and controlled comparisons of features, graph families, routing, and search budgets.

12 Conclusion

The direct-state graph computes output mismatch C and active-path U , then updates its addressed selectors by $S' = S \oplus (C \wedge U)$. Partial-prefix routing created more route keys in the tested lookup graph without improving held-out accuracy. The layered MUX model’s original rule accepts single-bit proposals that reduce global training error. Later joint CHANGE/USED masks and weighted J scores are separate experiments; on three matched logic tasks, the validation-selected $17E + Q$ variant reached 68.88% pooled held-out accuracy versus 70.10% for the original single-bit rule. The routed Boolean model’s principal rule is Eq. (29): it gates small candidate masks by active features and commits only a strict lexicographic improvement in route error, forgetting, and change cost. On its favorable binary-threshold task it reached 90.31% mean held-out accuracy versus 99.48% for the MLP, with smaller stored state and faster measured inference but slower training. These are distinct architectures and acceptance rules. Their training-set guarantees do not prove generalization.

References

- [1] R. E. Bryant, “Graph-Based Algorithms for Boolean Function Manipulation,” *IEEE Transactions on Computers*, C-35(8), pp. 677–691, 1986. <https://doi.org/10.1109/TC.1986.1676819>.
- [2] J. F. Miller and P. Thomson, “Cartesian Genetic Programming,” *European Conference on Genetic Programming*, LNCS 1802, pp. 121–132, 2000. https://doi.org/10.1007/978-3-540-46239-2_9.
- [3] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, “Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1,” 2016. <https://arxiv.org/abs/1602.02830>.